Day 7: Observability, Monitoring & Tracing

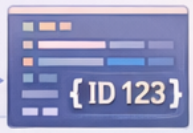
Understanding and measuring distributed systems in production.

Logs, Metrics, & Traces

- Logs capture events
- Metrics track system health
- Traces follow requests

Structured Logging & Correlation IDs

- Structured Logs
- Correlation IDs



Dashboards & Alerts

- Actionable Dashboards
- Alerting Rules

RED & USE Metrics

- RED: Request Rate
  - Error Rate - Duration
- USE: Utilization
  - Saturation, Errors

Dashboards & Alerts

- Actionable Dashboards
- Alerting Rules

Distributed Tracing

- Trace Requests Across Services
- Context Propagation

Sampling & Telemetry Costs

- Sampling Strategies
- Cardinality Trade-Offs

Black & White Box Monitoring

- **Black Box:** External Checks
- **White Box:** Internal Metrics

Observability helps detect failures and debug quickly

Day 7: Observability, Monitoring, and Tracing

Day 7 focuses on how distributed systems are measured, debugged, and understood once they are running in production.

It covers:

- logs, metrics, and traces
- structured logging and correlation IDs
- white-box vs black-box monitoring
- RED and USE metrics
- dashboards, alerting, and incident response
- distributed tracing and context propagation
- sampling, cardinality, and telemetry cost trade-offs

In short, Day 7 is about making production systems understandable enough to detect failures quickly and debug them without guesswork.

Core ideas

Logs, metrics, and traces

- `Logs` capture detailed events and are useful for debugging specific failures.
- `Metrics` summarize system behavior over time and are useful for dashboards and alerts.
- `Traces` connect work across services and help explain end-to-end latency.

Structured logging and correlation IDs

- `Structured logs` make machine filtering and aggregation easier than free-form log lines.
- `Correlation IDs` and `trace IDs` tie related events together across services.
- Without consistent request identifiers, debugging a distributed request becomes slow and error-prone.

Monitoring models and service health

- `Black-box monitoring` checks the externally visible behavior of a service.
- `White-box monitoring` inspects internals like queue depth, CPU, retries, and cache hit rate.
- Both are necessary because a service can look healthy internally while failing from the user's perspective.

RED, USE, and alerting

- `RED` stands for request rate, error rate, and duration.
- `USE` stands for utilization, saturation, and errors.
- Good alerts are actionable, low-noise, and tied to user impact or important resource limits.

Distributed tracing and telemetry trade-offs

- `Distributed tracing` follows one request across multiple services using propagated context.
- `Sampling` reduces telemetry cost, but can hide rare failures if configured badly.
- High-cardinality labels improve diagnosis, but they also increase storage and query cost.

Day-7 sample: Trace Context Propagator

This repository includes a small trace context propagation sample in both Python and Java.

Functional requirements

- Start a root trace for an incoming request
- Propagate trace headers between services
- Create a child span for each downstream call
- Emit correlated log lines that share the same trace ID

High-level components

- `Client`: initiates a request
- `Gateway`: starts the root trace
- `Service`: extracts incoming trace context and creates child spans
- `Headers`: carry trace identifiers between services
- `Logs`: expose the trace and span IDs for debugging

Data flow

1. A client request reaches the gateway
2. The gateway creates a root trace and logs the request
3. Trace headers are injected into the outbound call to the next service
4. The receiving service extracts the trace context and creates a child span
5. Every service logs with the same trace ID so the full path can be reconstructed

Project structure

```
Day-7/
  README.md
  trace-context-sample/
    python/
      trace_context.py
      demo.py
    java/
      TraceContext.java
      Main.java
```

Run the sample

Python

```
cd trace-context-sample\python
python demo.py
```

Java

```
cd trace-context-sample\java
javac Main.java TraceContext.java
java Main
```

Interview takeaways

- You cannot operate a distributed system reliably if you cannot correlate logs, metrics, and traces
- RED metrics are a strong default starting point for user-facing services
- Distributed tracing is valuable only if context propagation is consistent across every hop
- Alert quality matters more than alert quantity
- Telemetry is a product cost; sample and label carefully

Next improvements

- Add W3C `traceparent` header support
- Add span timing and latency measurement
- Export traces to OpenTelemetry-compatible format
- Add sampling policies for high-volume traffic
- Extend the sample into a small multi-service HTTP demo

Source Code Appendix

Day-7/trace-context-sample/python/trace_context.py

```python
from __future__ import annotations

from dataclasses import dataclass
from uuid import uuid4


@dataclass(frozen=True)
class TraceContext:
    trace_id: str
    span_id: str
    parent_span_id: str | None = None

    @staticmethod
    def start_root() -> "TraceContext":
        return TraceContext(trace_id=uuid4().hex, span_id=uuid4().hex[:16])

    @staticmethod
    def from_headers(headers: dict[str, str]) -> "TraceContext":
        trace_id = headers["x-trace-id"]
        span_id = headers["x-span-id"]
        parent_span_id = headers.get("x-parent-span-id") or None
        return TraceContext(trace_id=trace_id, span_id=span_id, parent_span_id=parent_span_id)

    def create_child(self) -> "TraceContext":
        return TraceContext(
            trace_id=self.trace_id,
            span_id=uuid4().hex[:16],
            parent_span_id=self.span_id,
        )

    def to_headers(self) -> dict[str, str]:
        headers = {
            "x-trace-id": self.trace_id,
            "x-span-id": self.span_id,
        }
        if self.parent_span_id is not None:
            headers["x-parent-span-id"] = self.parent_span_id
        return headers

    def format_log(self, service: str, message: str) -> str:
        parent = self.parent_span_id or "-"
        return (
            f"[service={service}] [trace_id={self.trace_id}] "
            f"[span_id={self.span_id}] [parent_span_id={parent}] {message}"
        )
```

Day-7/trace-context-sample/python/demo.py

```python
from trace_context import TraceContext


def gateway() -> None:
    root = TraceContext.start_root()
    print(root.format_log("api-gateway", "received client request /checkout"))

    order_span = root.create_child()
    print(order_span.format_log("api-gateway", "calling order-service"))
    order_service(order_span.to_headers())


def order_service(headers: dict[str, str]) -> None:
    incoming = TraceContext.from_headers(headers)
    print(incoming.format_log("order-service", "validated order payload"))

    payment_span = incoming.create_child()
    print(payment_span.format_log("order-service", "calling payment-service"))
    payment_service(payment_span.to_headers())


def payment_service(headers: dict[str, str]) -> None:
    incoming = TraceContext.from_headers(headers)
    print(incoming.format_log("payment-service", "authorized payment"))
```

```
if __name__ == "__main__":
    gateway()
```

Day-7/trace-context-sample/java/TraceContext.java

```
import java.util.HashMap;
import java.util.Map;
import java.util.UUID;

public record TraceContext(String traceId, String spanId, String parentSpanId) {
    public static TraceContext startRoot() {
        return new TraceContext(newId(), shortId(), null);
    }

    public static TraceContext fromHeaders(Map<String, String> headers) {
        return new TraceContext(
                headers.get("x-trace-id"),
                headers.get("x-span-id"),
                headers.get("x-parent-span-id")
        );
    }

    public TraceContext createChild() {
        return new TraceContext(traceId, shortId(), spanId);
    }

    public Map<String, String> toHeaders() {
        Map<String, String> headers = new HashMap<>();
        headers.put("x-trace-id", traceId);
        headers.put("x-span-id", spanId);
        if (parentSpanId != null) {
            headers.put("x-parent-span-id", parentSpanId);
        }
        return headers;
    }

    public String formatLog(String service, String message) {
        String parent = parentSpanId == null ? "-" : parentSpanId;
        return String.format(
                "[service=%s] [trace_id=%s] [span_id=%s] [parent_span_id=%s] %s",
                service,
                traceId,
                spanId,
                parent,
                message
        );
    }

    private static String newId() {
        return UUID.randomUUID().toString().replace("-", "");
    }

    private static String shortId() {
        return newId().substring(0, 16);
    }
}
```

Day-7/trace-context-sample/java/Main.java

```
public class Main {
    public static void main(String[] args) {
        gateway();
    }

    private static void gateway() {
        TraceContext root = TraceContext.startRoot();
        System.out.println(root.formatLog("api-gateway", "received client request /checkout"));

        TraceContext orderSpan = root.createChild();
        System.out.println(orderSpan.formatLog("api-gateway", "calling order-service"));
        orderService(orderSpan.toHeaders());
    }

    private static void orderService(java.util.Map<String, String> headers) {
        TraceContext incoming = TraceContext.fromHeaders(headers);
        System.out.println(incoming.formatLog("order-service", "validated order payload"));
```

```
        TraceContext paymentSpan = incoming.createChild();
        System.out.println(paymentSpan.formatLog("order-service", "calling payment-service"));
        paymentService(paymentSpan.toHeaders());
    }

    private static void paymentService(java.util.Map<String, String> headers) {
        TraceContext incoming = TraceContext.fromHeaders(headers);
        System.out.println(incoming.formatLog("payment-service", "authorized payment"));
    }
}
```